

Documento de Decisión Técnica: Sincronización Centralizada de Parámetros de Nómina (Multi-Cliente)

1. Contexto y Objetivo

Este documento presenta la evaluación técnica de tres opciones arquitectónicas para implementar una sincronización centralizada de parámetros de nómina en un entorno multi-cliente, donde PROGRESSA actúa como sistema central que distribuye datos maestros a múltiples instancias de Odoo cliente.

Contexto Técnico: - **Base de Desarrollo:** Odoo 17 - **Requisito:** La solución debe aprovechar las capacidades nativas de Odoo 17 pero ser flexible para futuras versiones (no version-dependent)

Objetivo: Seleccionar la arquitectura óptima que permita sincronizar parámetros de nómina hacia múltiples clientes de forma segura, escalable y mantenible, utilizando Odoo como base de desarrollo.

2. Opciones Evaluadas

Opción 1: Odoo Webhooks (Push desde PROGRESSA)

Sistema de notificaciones HTTP basado en eventos que permite a PROGRESSA (Odoo 17) enviar actualizaciones automáticas cuando ocurren cambios en los parámetros de nómina. Puede implementarse mediante: - **Módulos Odoo 17:** Utilizando Automated Actions o módulos de terceros - **Controllers personalizados:** Desarrollando endpoints HTTP en módulos Odoo 17 - **Integración con servicios externos:** Mediante cron jobs o triggers de base de datos

Opción 2: Servicio API Propio (REST/GraphQL) - Desarrollado en Odoo 17

API personalizada desarrollada como módulo Odoo 17 que puede operar en modo pull (clientes consultan) o push (PROGRESSA envía). Implementación mediante: - **Controllers REST en Odoo 17:** Utilizando `@http.route` decorators - **JSON-RPC nativo de Odoo 17:** Aprovechando la API JSON-RPC integrada - **Módulos personalizados:** Desarrollando endpoints específicos para sincronización - **Message Queue integrado:** Usando Odoo's job queue o integración con RabbitMQ/Celery

Opción 3: Odoo JSON-RPC/XML-RPC (Push desde PROGRESSA)

Protocolo nativo de Odoo 17 que permite comunicación remota. Odoo 17 soporta tanto JSON-RPC (recomendado) como XML-RPC (legacy): - **JSON-RPC:** Protocolo moderno y eficiente, nativo en Odoo 17 - **XML-RPC:** Protocolo legacy aún soportado pero no recomendado

3. Comparativa Detallada

3.1 Tabla Comparativa de Criterios

Criterio	Odoo Webhooks	API Propia (REST/GraphQL)	Odoo XML-RPC
Seguridad (Autenticación/Autorización)	Media • Autenticación mediante tokens en headers HTTP • Depende de la implementación del módulo de webhooks en Odoo 17 • Requiere SSL/TLS para producción • Validación de firma opcional pero recomendada • Limitado control granular por cliente • Puede usar API Keys de Odoo 17	Alta • Control total sobre autenticación (OAuth2, JWT, API Keys) • Aprovecha sistema de usuarios y grupos de Odoo 17 • Autorización granular mediante <code>ir.model.access</code> de Odoo • Rate limiting personalizado en controllers • Auditoría completa usando logging de Odoo • Soporte para mTLS (mutual TLS) • Implementación de políticas de seguridad propias • Integración con <code>res.users</code> y <code>res.groups</code> de Odoo 17	Media (JSON-RPC) / Baja (XML-RPC) • JSON-RPC: Autenticación mediante usuario/contraseña Odoo 17 • XML-RPC: Mismo método pero menos seguro • Depende de permisos de usuario en Odoo 17 (<code>ir.model.access</code>) • Puede usar API Keys de Odoo 17 (JSON-RPC) • Menos flexible para políticas de seguridad complejas • Credenciales transmitidas en cada request (mitigado con HTTPS)
Escalabilidad	Alta (5-50 clientes) Media (500+ clientes) • Excelente para 5-50 clientes • Event-driven: bajo overhead por cliente • Puede requerir cola de mensajes (RabbitMQ/Kafka) para 500+ • Cada webhook es independiente • Escalabilidad horizontal posible con load balancer	Muy Alta (5-500+ clientes) • Diseñada para escalar horizontalmente • Load balancing nativo • Caching de respuestas (Redis) • Rate limiting por cliente • GraphQL reduce carga con consultas específicas • Arquitectura stateless facilita escalado • Soporte para CDN y edge computing	Baja-Media • Conexiones síncronas pueden saturar servidor • Sin soporte nativo para rate limiting • Overhead de XML limita throughput • Dificulta escalado horizontal • Mejor para <50 clientes simultáneos

Criterio	Odoo Webhooks	API Propia (REST/GraphQL)	Odoo XML-RPC
Manejo de Errores (Cliente Offline)	Complejo • Requiere cola de reintentos (dead letter queue) • Necesita almacenamiento temporal de eventos fallidos • Timeout y reintentos configurables • Puede perder eventos si no hay persistencia • Requiere monitoreo de estado de clientes • Necesita mecanismo de reconciliación	Robusto • Cola de mensajes con persistencia (RabbitMQ/SQS) • Reintentos exponenciales configurables • Dead letter queue para eventos fallidos • Logging y alertas centralizados • Estado de sincronización por cliente • Mecanismo de reconciliación programada • Puede implementar circuit breaker pattern	Limitado • Reintentos manuales o mediante script • Sin persistencia nativa de eventos fallidos • Depende de implementación custom • Dificulta tracking de estado • Requiere monitoreo externo
Facilidad de Mantenimiento y Despliegue	Alta • Configuración mediante UI de Odoo 17 • Módulo Odoo 17: despliegue estándar • Actualizaciones de Odoo 17 pueden afectar funcionalidad • Logs integrados en Odoo 17 (<code>_logger</code>) • Debugging con herramientas de Odoo 17 • Compatible con sistema de módulos de Odoo • Versionado mediante <code>__manifest__.py</code>	Alta (si es módulo Odoo) • Desarrollo como módulo Odoo 17: despliegue estándar • Versionado mediante <code>__manifest__.py</code> y versionado de API • CI/CD integrado con flujo de Odoo • Documentación en módulo Odoo • Testing con <code>odoo.tests.common</code> • Mayor control pero dentro del ecosistema Odoo • Puede usar tecnologías modernas (Docker, K8s) para servicios externos	Alta • Sin código adicional (usa funcionalidad nativa Odoo 17) • JSON-RPC: debugging más simple (formato JSON) • XML-RPC: debugging complejo (XML verbose) • Scripts de sincronización como módulos Odoo • Mantenimiento dentro del ecosistema Odoo 17 • Compatible con sistema de módulos

Criterio	Odoo Webhooks	API Propia (REST/GraphQL)	Odoo XML-RPC
Bidireccionalidad (Reporte Éxito/Fallo)	Limitada • Webhook es unidireccional (PROGRESSA → Cliente) • Cliente puede responder con HTTP status, pero no garantizado • Requiere endpoint callback adicional para confirmación • Sin garantía de entrega (fire-and-forget) • Necesita polling o webhook inverso para confirmación	Completa • Respuestas HTTP estándar (200, 400, 500, etc.) • Endpoints específicos para confirmación • Webhooks de respuesta opcionales • Estado de sincronización consultable • Logs bidireccionales • Puede implementar ACK/NACK explícitos	Parcial • Respuesta síncrona en la llamada • Pero sin mecanismo robusto de confirmación • Depende de interpretación de respuesta XML • Sin estado persistente de confirmación
Performance	Alta • Event-driven: solo cuando hay cambios • Bajo overhead por evento • JSON ligero • Asíncrono (no bloquea operaciones)	Muy Alta • REST: eficiente con JSON • GraphQL: solo datos necesarios • Caching agresivo posible • Compresión HTTP • Optimización de queries	Baja • XML verboso (mayor tamaño) • Parsing más costoso • Síncrono (puede bloquear) • Sin compresión nativa

4. Identificación del "Maestro" (Source of Truth)

4.1 Respuesta a Criterios de Aceptación

¿Dónde vivirá la data "maestra"?

La data maestra vivirá en **PROGRESSA**, que será una **instancia Odoo 17 completa** funcionando como sistema centralizado. Los datos se almacenarán en la **base de datos PostgreSQL** de esta instancia Odoo 17, utilizando los modelos estándar y personalizados del módulo `l10n_do_hr_payroll` y sus dependencias.

¿Será una instancia de Odoo "madre" o una base de datos/API independiente?

- **Instancia Odoo 17 "madre"** (Recomendada)
- PROGRESSA será una instancia Odoo 17 completa y funcional
- Los parámetros maestros se gestionan directamente en Odoo 17 usando la interfaz estándar
- La API de sincronización se desarrolla como módulo Odoo 17 dentro de la misma instancia

- **Ventajas:** Aprovecha todas las capacidades de Odoo (ORM, seguridad, UI, etc.), no requiere infraestructura adicional, desarrollo dentro del ecosistema Odoo
- **Base de datos/API independiente**
 - Requeriría desarrollo de API desde cero
 - Necesitaría infraestructura adicional
 - Duplicaría funcionalidades que Odoo ya provee
 - Mayor complejidad de mantenimiento

¿De dónde saldrán los datos?

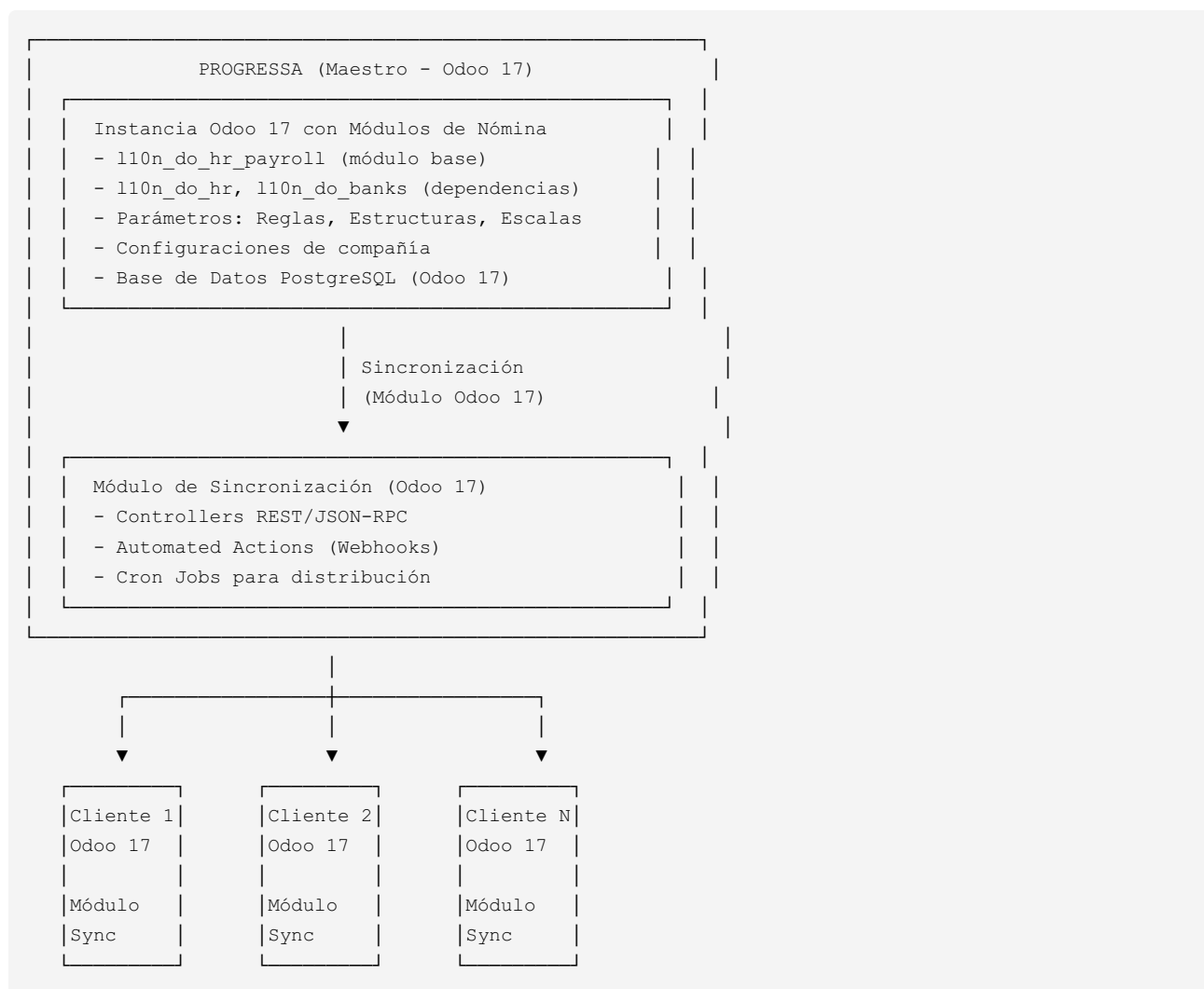
Los datos saldrán directamente de los modelos de Odoo 17 en la instancia PROGRESSA:

- **Modelos de Odoo 17:** Los parámetros se almacenan en modelos como `hr.salary.rule`, `l10n.do.hr.retention.scale`, `l10n.do.hr.payroll.payment.division`, etc.
- **Base de datos PostgreSQL:** Los datos residen en las tablas de PostgreSQL de la instancia Odoo 17
- **Módulos de Nómina:** Los datos provienen de los módulos instalados en PROGRESSA:
 - `l10n_do_hr_payroll` - Módulo principal de nómina República Dominicana
 - `l10n_do_hr` - Localización de empleados
 - `l10n_do_banks` - Bancos de República Dominicana
- **Módulos dependientes:** `hr_payroll`, `hr_contract`, `hr_payroll_account`
- **Origen de los datos:** Los usuarios administradores en PROGRESSA crean y modifican los parámetros usando la interfaz estándar de Odoo 17. El módulo de sincronización detecta estos cambios y los distribuye a los clientes.

Resumen de la Decisión:

Criterio	Decisión
Ubicación de datos maestros	Instancia Odoo 17 PROGRESSA
Tipo de sistema	Instancia Odoo 17 "madre" (no API independiente)
Base de datos	PostgreSQL de la instancia Odoo 17
Origen de datos	Modelos Odoo 17 del módulo <code>l10n_do_hr_payroll</code>
Gestión de datos	Interfaz estándar de Odoo 17
API de sincronización	Módulo Odoo 17 dentro de la misma instancia

4.2 Arquitectura del Maestro



4.4 Parámetros Maestros a Sincronizar

Basado en la estructura del proyecto `odoo-pro`, los siguientes modelos contienen parámetros maestros que deben sincronizarse:

Modelos Principales de Parámetros:

1. `hr.salary.rule` - Reglas de salario
2. Campos personalizados: `l10n_do_is_news`, `l10n_do_type_news`
3. Configuración de cálculos de nómina
4. Categorías y secuencias
5. `hr.payroll.structure` - Estructuras de nómina
6. Configuración de estructuras salariales
7. Asociación con reglas de salario
8. `l10n.do.hr.retention.scale` - Escalas de retención (ISR)
9. Escalas anuales de retención
10. Montos base, topes, porcentajes

11. Campos: `name`, `sequence`, `exempt`, `percent`, `base_amount`, `top_amount`
12. `l10n.do.hr.payroll.payment.division` - División de pagos
13. Parámetros por frecuencia de pago
14. Campos: `name` (frecuencia), `payment_division`, `company_id`
15. Constraint único por compañía y frecuencia
16. `l10n.do.hr.payroll.days.division` - División de días
17. Parámetros de división de días por frecuencia
18. Campos: `name` (frecuencia), `days_division`, `company_id`
19. `l10n.do.occupational.risk.type` - Tipos de riesgo ocupacional
20. Tipos de riesgo y porcentajes
21. Campos: `name`, `percentage`
22. `res.company` - Configuraciones de compañía relacionadas con nómina
23. `l10n_do_hr_dependent_amount` - Monto por dependiente
24. `l10n_do_ngo` - Indicador de ONG
25. `l10n_do_occupational_risk_type_id` - Tipo de riesgo ocupacional
26. `l10n_do_last_payroll_day` - Día del último pago de nómina
27. Relaciones con `l10n_do_hr_payroll_payment_division_ids` y `l10n_do_hr_payroll_days_division_ids`
28. `hr.payroll.structure.type` - Tipos de estructura de nómina
29. Configuración de tipos de estructura
30. `hr.salary.rule.category` - Categorías de reglas de salario
31. Categorización de reglas

Modelos de Datos Maestros (data files): - Configuraciones iniciales en archivos XML de data - Valores por defecto y catálogos base

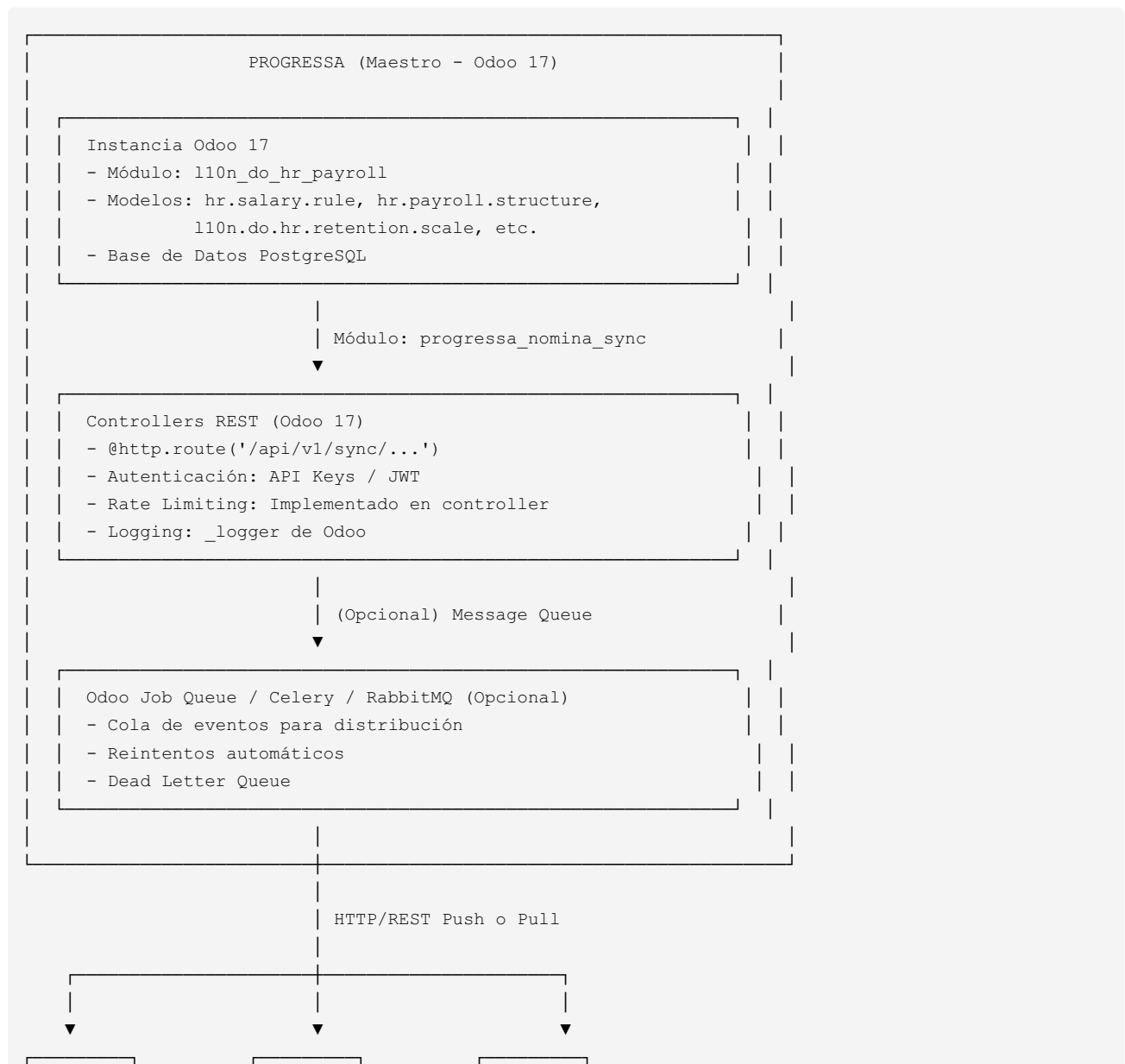
4.5 Consideraciones del Maestro

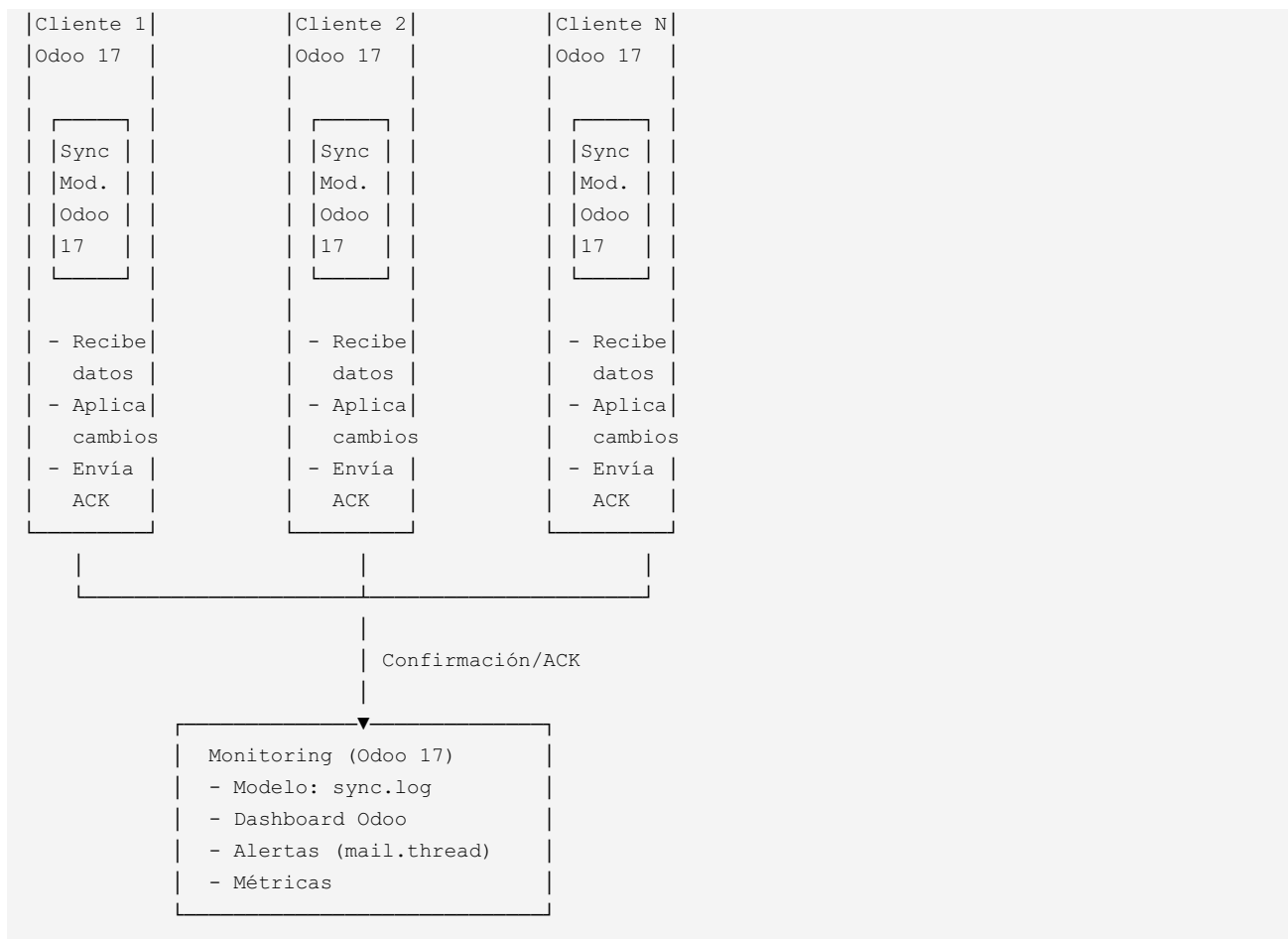
- **Ubicación:** PROGRESSA será la única fuente de verdad (Single Source of Truth - SSOT)
- **Implementación en Odoo 17:**
 - Modelos de datos: Modelos del módulo `l10n_do_hr_payroll` y dependencias
 - Módulo de sincronización: Desarrollar nuevo módulo siguiendo estructura de `odoo-pro`
 - Base de datos: PostgreSQL estándar de Odoo 17
 - API: Controllers REST desarrollados como parte del módulo de sincronización
 - Estructura: Seguir convenciones de nombres y organización de `odoo-pro` (`l10n_do_*`)
- **Responsabilidades:**
 - Gestión centralizada de parámetros de nómina en modelos Odoo 17
 - Validación de datos antes de sincronización (usando constraints existentes en modelos)
 - Versionado de cambios (mediante `mail.tracking` o campos `write_date`)

- Auditoría de modificaciones (logs de Odoo 17 usando `_logger`)
- Control de acceso y permisos (`ir.model.access` , `ir.rule`)
- Respetar constraints existentes (ej: `unique_parameterization` en `payment_division`)
- **Ventajas:**
 - Consistencia garantizada entre clientes
 - Facilita actualizaciones masivas de parámetros
 - Simplifica mantenimiento y soporte
 - Permite rollback centralizado
 - Aprovecha todas las capacidades nativas de Odoo 17
 - Desarrollo dentro del ecosistema Odoo (no requiere servicios externos)
 - Compatible con estructura existente del proyecto `odoo-pro`

5. Flujo de Datos - Arquitectura Recomendada

5.1 Diagrama de Flujo: Opción Recomendada (API REST desarrollada en Odoo 17)





5.2 Flujo Detallado de Sincronización (Implementado en Odoo 17)

Modo Push (Recomendado para actualizaciones inmediatas):

1. **Cambio en PROGRESSA (Odoo 17):** Usuario modifica parámetro de nómina en modelos como `hr.salary.rule`, `l10n.do.hr.retention.scale`, `l10n.do.hr.payroll.payment.division`, etc.
2. **Trigger/Override:** Método `write()` o `create()` detecta cambio (override en modelos del módulo de sincronización)
3. **Evento Generado:** Módulo de sincronización genera evento identificando modelo y registro modificado
4. **Controller REST:** Endpoint `/api/v1/sync/push` recibe evento, valida datos
5. **Message Queue (Opcional):** Evento encolado usando Odoo Job Queue o Celery
6. **Distribución:** Worker procesa cola y envía HTTP POST a clientes activos
7. **Cliente Recibe (Odoo 17):** Módulo sync en cliente recibe actualización vía controller
8. **Aplicación:** Módulo aplica cambios usando ORM de Odoo 17 `create()`, `write()`
9. **Confirmación:** Cliente envía ACK HTTP a endpoint `/api/v1/sync/ack` en PROGRESSA
10. **Registro:** Estado de sincronización actualizado en modelo `sync.log` en PROGRESSA

Modo Pull (Alternativa para clientes con restricciones de red):

1. **Cliente Solicita (Odoo 17):** Cron job en módulo sync hace polling periódico
2. **API Responde:** Endpoint `/api/v1/sync/pull` retorna cambios desde última sync
3. **Cliente Aplica:** Módulo sync aplica cambios usando ORM de Odoo 17
4. **Confirmación:** Cliente confirma recepción exitosa actualizando timestamp local

5.3 Componentes Clave (Desarrollados en Odoo 17)

- **Controllers REST (Odoo 17):** Punto de entrada, desarrollados como parte del módulo
 - Autenticación mediante API Keys almacenadas en `res.users.api_key`
 - Rate limiting implementado en decoradores personalizados
 - Validación usando constraints y métodos de Odoo 17
 - **Módulo de Sincronización (Odoo 17):** Lógica de negocio
 - Modelos: `progressa.sync.log`, `progressa.sync.client` (nuevos modelos para tracking)
 - Validación usando `@api.constrains` y `@api.onchange`
 - Transformación de datos entre instancias Odoo 17
 - Mapeo de modelos maestros: `hr.salary.rule`, `l10n.do.hr.retention.scale`, `l10n.do.hr.payroll.payment.division`, etc.
 - Respeto a constraints existentes en modelos del proyecto `odoo-pro`
 - **Message Queue (Opcional):** Garantiza entrega
 - Odoo Job Queue (módulo `queue_job`) o
 - Celery con integración Odoo o
 - RabbitMQ con workers Python
 - **Módulo Sync en Cliente (Odoo 17):** Componente en cada cliente
 - Módulo ligero que recibe y aplica sincronizaciones
 - Utiliza ORM de Odoo 17 para aplicar cambios
 - Cron jobs para modo pull
 - **Monitoring (Odoo 17):** Dashboard integrado
 - Modelo `progressa.sync.log` para tracking
 - Vistas Odoo 17 para dashboard
 - Alertas mediante `mail.thread` y notificaciones
-

6. Recomendación Final

6.1 Arquitectura Seleccionada: API REST desarrollada como Módulo Odoo 17 con Message Queue Opcional

6.2 Justificación de la Selección

Ventajas Clave sobre Odoo Webhooks:

1. **Seguridad Superior:**
2. Control total sobre autenticación y autorización
3. Implementación de políticas de seguridad específicas por cliente

4. Mejor auditoría y trazabilidad

5. Escalabilidad Comprobada:

6. Arquitectura diseñada para 500+ clientes desde el inicio

7. Escalado horizontal sin límites técnicos

8. Message queue garantiza distribución eficiente

9. Manejo de Errores Robusto:

10. Dead letter queue para eventos fallidos

11. Reintentos automáticos con backoff exponencial

12. Reconciliación programada para clientes offline

13. Estado de sincronización persistente

14. Bidireccionalidad Completa:

15. Confirmación explícita de recepción (ACK/NACK)

16. Endpoints para consulta de estado

17. Logs bidireccionales completos

18. Desarrollo en Odoo 17:

19. Desarrollo como módulo Odoo 17 estándar

20. Aprovecha todas las capacidades nativas (ORM, seguridad, logging)

21. Compatible con ecosistema Odoo (módulos, apps, etc.)

22. Despliegue estándar de módulos Odoo

23. Version-independent: código diseñado para ser compatible con futuras versiones

24. Flexibilidad de Implementación:

25. Puede combinar push y pull según necesidades del cliente

26. GraphQL permite consultas eficientes

27. Versionado de API para evolución gradual

Desventajas y Mitigaciones:

1. Mayor Complejidad de Desarrollo:

2. **Mitigación:** Desarrollo como módulo Odoo 17 estándar (no requiere frameworks externos)

3. **Mitigación:** Reutilización de componentes nativos de Odoo 17 (ORM, seguridad, etc.)

4. **Mitigación:** Uso de decoradores y helpers de Odoo 17

5. Mantenimiento Propio:

6. **Mitigación:** Documentación en módulo Odoo 17 (README, docstrings)

7. **Mitigación:** Monitoreo integrado en Odoo 17 (logs, métricas)

8. Tiempo de Desarrollo Inicial:

9. **Mitigación:** Desarrollo incremental (MVP primero)
10. **Mitigación:** Aprovechar estructura estándar de módulos Odoo 17
11. **Mitigación:** Reutilizar código de otros módulos Odoo existentes

6.3 Por qué NO las otras opciones:

Odoo Webhooks:

- **Limitaciones de seguridad:** Depende de implementación del módulo
- **Escalabilidad dudosa:** Requiere infraestructura adicional para 500+ clientes
- **Manejo de errores limitado:** Sin persistencia nativa de eventos fallidos
- **Bidireccionalidad incompleta:** Fire-and-forget sin garantías

Odoo XML-RPC:

- **Tecnología legacy:** No recomendado para nuevos desarrollos
- **Performance limitada:** XML verboso y parsing costoso
- **Escalabilidad baja:** Conexiones síncronas limitan throughput
- **Mantenimiento complejo:** Debugging difícil, dependencia de Odoo

6.4 Arquitectura Híbrida Recomendada (Desarrollada en Odoo 17)

Para maximizar beneficios, se recomienda una **arquitectura híbrida desarrollada como módulos Odoo 17**:

- **API REST** como componente principal (controllers en módulo Odoo 17)
 - **Message Queue (Opcional):** Odoo Job Queue o Celery para distribución confiable
 - **Webhooks opcionales:** Implementados mediante Automated Actions de Odoo 17
 - **Modo Pull:** Cron jobs de Odoo 17 para clientes con restricciones de firewall
 - **JSON-RPC nativo:** Como alternativa para clientes que prefieren protocolo nativo de Odoo
-

7. Recursos y Referencias

Documentación Técnica

- **REST API Best Practices:**
 - <https://restfulapi.net/>
 - <https://www.restapitutorial.com/>
- **GraphQL Documentation:**
 - <https://graphql.org/learn/>
- **Message Queue Patterns:**
 - RabbitMQ: <https://www.rabbitmq.com/documentation.html>
 - AWS SQS: <https://docs.aws.amazon.com/sqs/>

- **Odoo 17 External API:**

- https://www.odoo.com/documentation/17.0/developer/reference/external_api.html

- <https://www.odoo.com/documentation/17.0/developer/reference/backend/orm.html>

- **Odoo 17 JSON-RPC:**

- https://www.odoo.com/documentation/17.0/developer/reference/external_api.html#json-rpc

Frameworks y Tecnologías Recomendadas (dentro de Odoo 17)

- **Desarrollo Odoo 17:**

- Controllers HTTP nativos de Odoo 17

- ORM de Odoo 17 para acceso a datos

- Sistema de módulos estándar de Odoo 17

- **Message Queue (Opcional):**

- Odoo Job Queue: <https://github.com/OCA/queue>

- Celery con integración Odoo

- RabbitMQ con workers Python

9. Conclusión

La **API REST desarrollada como Módulo Odoo 17 con Message Queue Opcional** es la solución óptima para la sincronización centralizada de parámetros de nómina en un entorno multi-cliente, ofreciendo:

- Seguridad robusta y escalable (aprovechando sistema nativo de Odoo 17)
- Manejo de errores confiable (con Message Queue opcional)
- Bidireccionalidad completa (ACK/NACK implementados)
- Escalabilidad desde 5 hasta 500+ clientes
- Desarrollo dentro del ecosistema Odoo 17 (módulos estándar)
- Flexibilidad de implementación (push/pull, REST/JSON-RPC)
- Version-independent: código diseñado para compatibilidad futura
- Aprovecha todas las capacidades nativas de Odoo 17 (ORM, seguridad, logging, testing)

Esta arquitectura proporciona la base sólida necesaria para un sistema de sincronización empresarial que crecerá con las necesidades del negocio, manteniendo la coherencia con el ecosistema Odoo y facilitando el mantenimiento a largo plazo.
